

FDS 复习—哈希表 (Hashing)

6 月 29 日冲刺

1 核心思想

不用比较，直接算出位置。

- 查找: $O(1)$
- 插入: $O(1)$
- 跟数据量无关

考点

基于比较的查找下界是 $O(\log N)$ (二分查找)。哈希通过**不比较**突破了它。

2 哈希表术语

术语	含义
b (buckets)	桶的数量
s (slots)	每个桶几个槽
n	当前表中元素个数
T	所有可能 key 的总数
$\lambda = n/(s \cdot b)$	负载密度 (Loading Density)

- **冲突 (Collision)**: 两个不同 key 算出了同一个桶
- **溢出 (Overflow)**: key 算出的桶已经满了
- 当 $s=1$ 时, 冲突 = 溢出 (同时发生)

3 哈希函数

要求: 计算快 + 分布均匀 (无偏)。

3.1 整数键

```
1 h(x) = x % TableSize
```

考点

TableSize 必须是质数！否则后几位相同的数扎堆。

3.2 字符串键—Hash3（推荐）

```
1 int hash(const char* key, int TableSize) {
2     unsigned int h = 0;
3     while (*key)
4         h = (h << 5) + *key++;    // h*32 + 下一字符
5     return h % TableSize;
6 }
```

为什么用 32? $32 = 2^5$ ，移位比乘法快。

4 冲突解决方案一：分离链接法

思想：每个桶挂一个链表。

```
1 typedef struct Node { int key; struct Node* next; } Node;
2 Node* table[T];
3
4 void insert(int key) {
5     int h = key % T;
6     Node* node = malloc(sizeof(Node));
7     node->key = key;
8     node->next = table[h];    // 头插法
9     table[h] = node;
10 }
11
12 Node* find(int key) {
13     Node* p = table[key % T];
14     while (p && p->key != key) p = p->next;
15     return p;
16 }
```

特点：简单，但需要额外链表空间。

5 冲突解决方案二：开放地址法

思想：冲突了就找下一个空位。

探测序列： $h_i = (\text{hash}(\text{key}) + f(i)) \bmod \text{TableSize}$

5.1 线性探测 (Linear Probing)

```
1 f(i) = i          // 每次 +1
```

优点：简单 缺点：一次聚集 (Primary Clustering)，冲突的人扎堆

5.2 二次探测 (Quadratic Probing)

```
1 f(i) = i^2        // +1, +4, +9, +16...
```

考点

定理：如果 TableSize 是质数且表至少一半空，二次探测一定能找到空位。

为什么？前 TableSize/2 个探测值 $i^2 \bmod \text{TableSize}$ 全是不同的。

缺点：二次聚集 (Secondary Clustering) — 相同 hash 的人探测路径一样

5.3 双哈希 (Double Hashing)

```
1 f(i) = i * h2(key) // 步长取决于第二个哈希函数
```

推荐： $h_2(x) = R - (x \% R)$ ，其中 R 是比 TableSize 小的质数（如 7）。

特点：基本消除聚集，但计算稍多。

5.4 代码框架（二次探测）

```
1 int find(int table[], int key, int T) {
2     int h = key % T;
3     for (int i = 0; i <= T/2; i++) {    // 最多探一半
4         int pos = (h + i*i) % T;
5         if (table[pos] == EMPTY) return -1;    // 没有
6         if (table[pos] == key) return pos;    // 找到了
7     }
8     return -1;
9 }
```

6 再哈希 (Rehashing)

为什么：负载密度太高 → 查找变慢。

怎么做：

1. 建一个更大的表（约 2 倍，且是质数）
2. 把旧表所有元素重新哈希插入新表
3. 释放旧表

触发时机：

- 插入失败（没空位了）
- 负载密度超过阈值（如 0.5）
- 查找性能严重下降

考点

摊还分析：Rehashing 一次 $O(N)$ ，但摊还到每次插入是 $O(1)$ 。

7 分离链接 vs 开放地址

	分离链接	开放地址
实现	链表	纯数组
空间	额外指针	无额外开销
满后	无上限（链表可增长）	Rehashing
缓存	差（内存不连续）	好（连续内存）